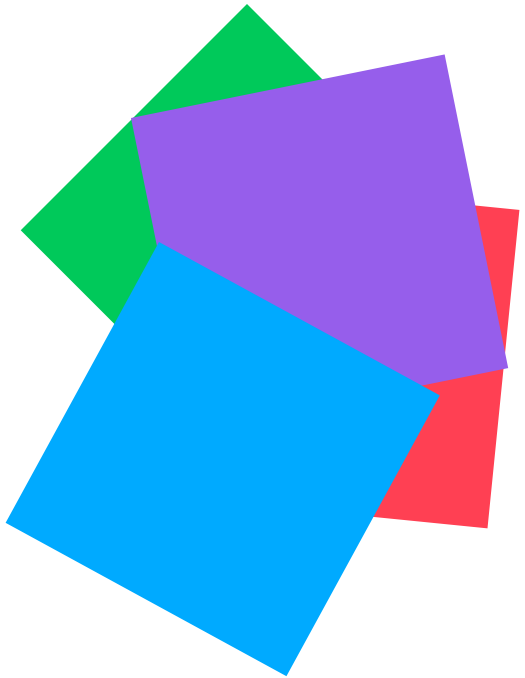
Конкурентность

Разработка микросервисов на Golang



План

01 Фундаментальные концепты конкурентности

02 Что такое горутины и каналы

03 Примитивы синхронизации, тикеры и таймеры

04 Типичные задачи на конкурентность

I. Фундаментальные концепты

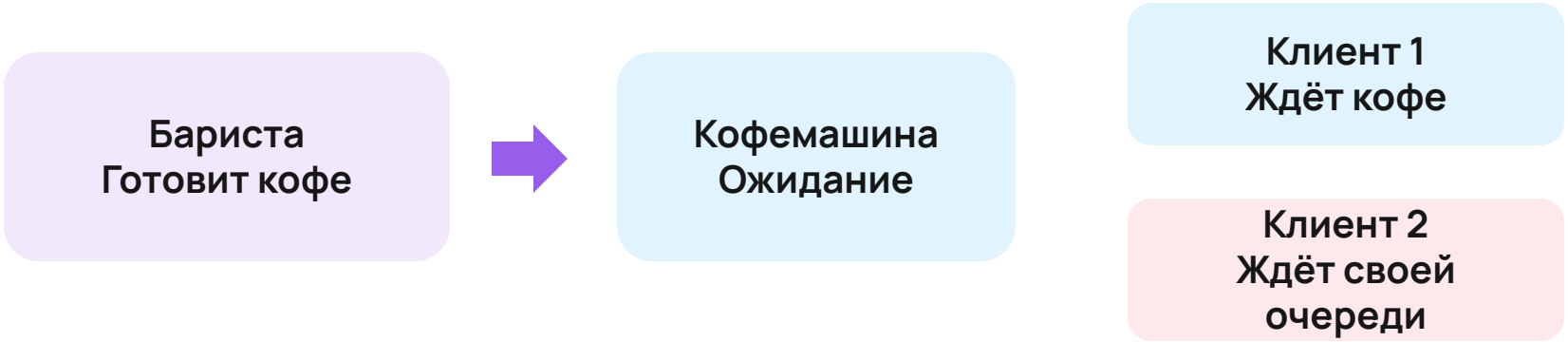
01 Многопоточность и асинхронность

02 Реализация в Go

03 Проблема традиционных подходов

Зачем нужна конкуретность?

Один сотрудник ждёт приготовления и блокирует очередь



Пока готовится кофе, следующий заказ не обслуживается.
Асинхронная работа позволяет использовать время ожидания.

Асинхронность vs параллелизм

Асинхронность: координируем ожидание

Кофемашина готовит кофе

Бариста продаёт десерт

Один бариста переключается на другую задачу, пока машина работает.
Несколько бариста не обязательны.

Параллелизм: выполняем одновременно

Бариста 1 готовит кофе

Бариста 2 продаёт десерт

Два сотрудника выполняют разные задачи в один момент.
Нужны независимые исполнители.

Различия тезисно

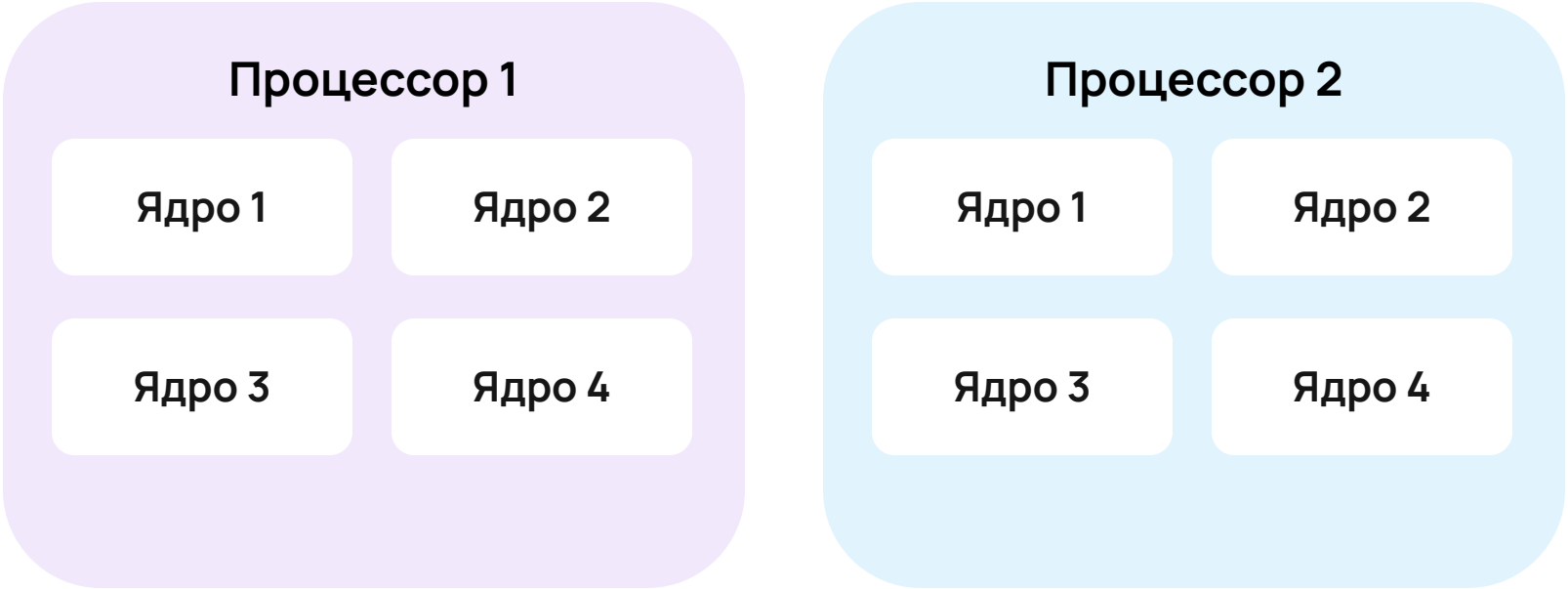
Асинхронность

- Одна задача приостанавливается, чтобы выполнить другую
- Возможна реализация на одном физическом ядре
- Требует диспетчера
- Необходимо переключать контекст
- Важна при I/O и сетевых операциях

Параллелизм

- Несколько задач выполняются одновременно
- Требует несколько физических ядер
- Позволяет производить вычисления параллельно
- Важен при CPU-bound задачах

Топология процессора



Несколько физических ядер позволяют выполнять вычисления параллельно.

Зачем это в современном программировании

- Аппаратные ограничения: частоты не растут, растёт число ядер
- I/O-задержки: конкурентность позволяет не простаивать во время ожидания
- Реактивность: одновременная обработка множества событий

Конкурентность — это не про скорость, а про эффективное использование времени

Примеры

- Веб-сервер: обрабатывает тысячи запросов одновременно
- UI-приложение: не блокирует интерфейс во время загрузки
- Микросервисы: взаимодействуют через сети — естественно асинхронно

Проблемы традиционных подходов

01 Потоки ОС

- Тяжеловесные: потребляют мегабайты памяти, требуют переключения контекста ОС
- Ограничены по количеству (не масштабируются до 100k задач)

02 Ручные блокировки

- Дедлоки, гонки, тяжело отслеживать

03 Callback / async

- Пирамида вызовов в коде
- Потеря линейного потока управления

Закон Амдала

Ускорение ограничено последовательной частью программы

$$S(N) = 1 / ((1 - P) + P / N)$$

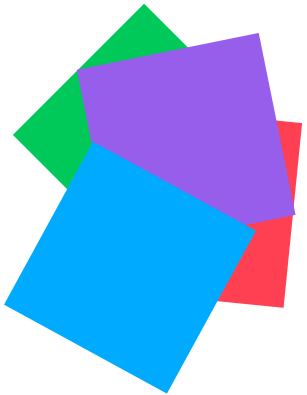
S(N) — теоретическое ускорение

P — доля параллельной работы

1 – P — последовательная доля

N — число процессоров (ядер)

Потренируемся



Представьте ситуацию

Вы приходите в МФЦ, берёте талон, подходите к окну, вам выдают справку. Одновременно с этим ваш друг проходит такой же путь и получает справку в соседнем окне. Какой это вид конкурентности?

1. Асинхронность
2. Параллелизм

Вопрос

Вы приходите в ресторан, в котором один официант на все столики.

Это:

1. Асинхронность
2. Параллельность

Выводы первого блока

Конкуренция — декомпозиция задачи на независимые процессы или потоки, которые могут выполняться независимо

Параллелизм — физическая возможность исполнять их одновременно

II. Горутины и каналы

01 Что такое горутины и каналы?

02 Управление и завершение горутин

Что такое горутина?

Горутина — лёгкая конкурентная задача, управляемая рантаймом Go, а не операционной системой

Запуск горутины

Без ожидания программа завершится до выполнения горутин

```
func main() {
   go task("A")
   go task("B")
   time.Sleep(time.Second)
}

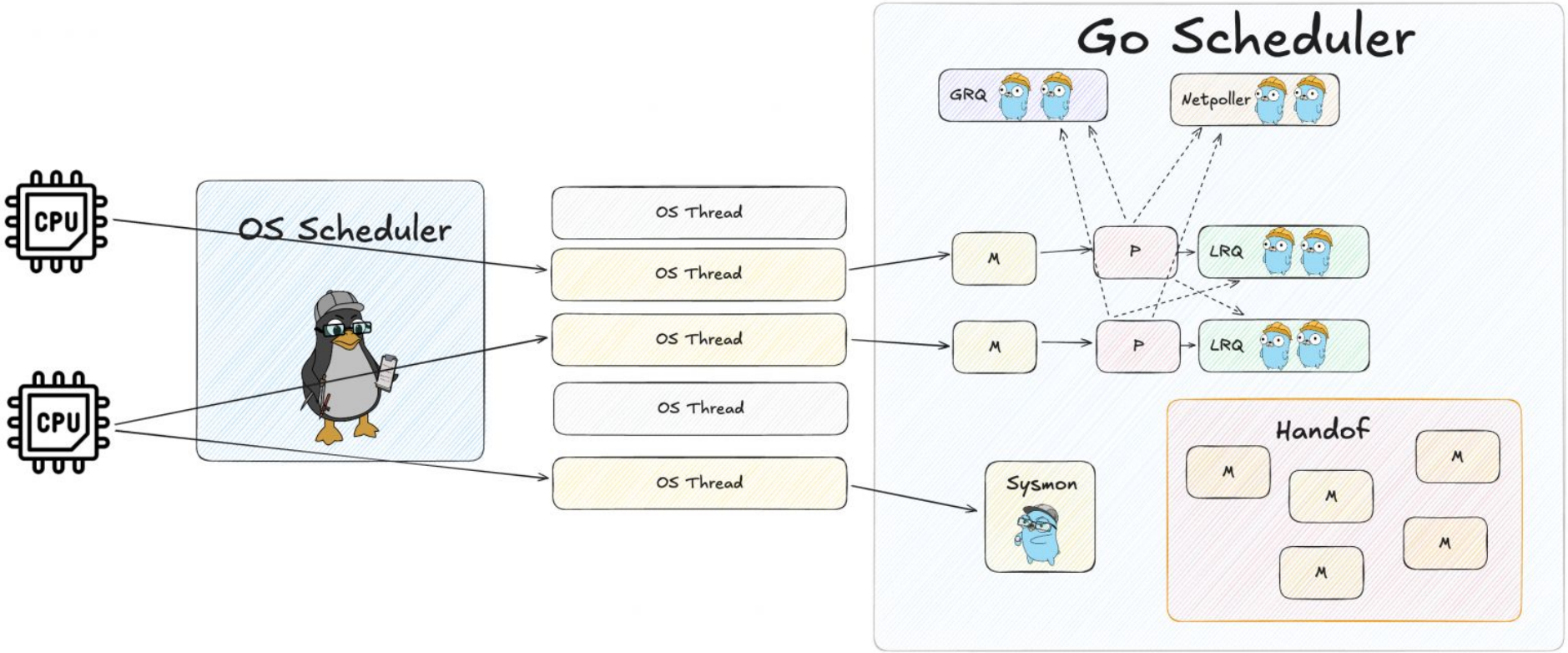
func task(name string) {
   for i := 0; i < 3; i++ {
      fmt.Println(name, i)
   }
}
```

Основные особенности

- Запускается через Go `func() {...}()`
- Имеет собственный стек (начинается с ~2 KB)
- Может быть миллионы горутин в одной программе
- Управляется Go-планировщиком, а не ОС

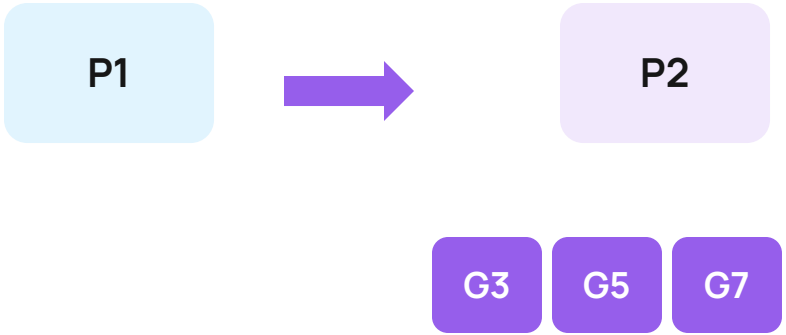
Поток ОС	Горутина
Дорогое создание	Создание дешёвое
Тяжелое переключение контекста	Быстрый и кооперативный
Управляется ОС	Управляется Go runtime

G-M-P модель


Go Scheduler
GRQ
Netpoller
CPU
CPU
OS Scheduler
OS Thread
OS Thread
OS Thread
OS Thread
OS Thread
M
P
LRQ
M
P
LRQ
Sysmon
Handof
M
M
M
M
M

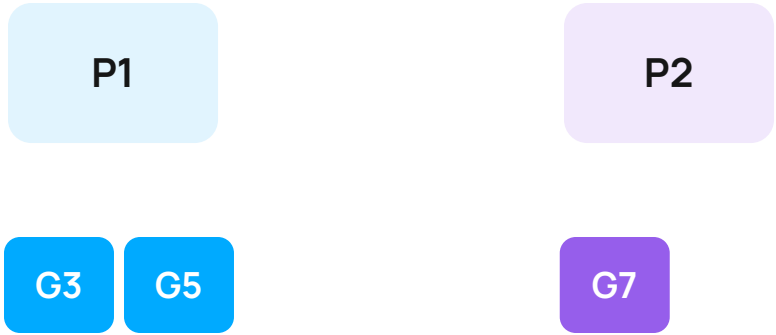
Work stealing

P1 свободен, у P2 есть очередь



Пустая локальная очередь → ищем работу у другого P.

После переноса работы



P1 выполняет G3; G5 остаётся в его очереди.

Парковка горутины и syscalls

Парковка — это временная приостановка выполнения горутины, когда она не может продолжать работу (например, ждёт канал, sync.Mutex, WaitGroup, Cond, time.Sleep, и т. д.).

При парковке горутина не блокирует поток ОС, а меняет свой статус и снимается с потока — поток выполняет другие горутины)

При блокировке системными вызовами (сеть, чтение файлов и т.д.) блокируется поток M (системный)

Рантайм может создать больше потоков M (например, если часть заблокирована syscall)

Go Netpoller

Асинхронный I/O внутри планировщика

Схема взаимодействия:
Горутина → syscalls → netpoller → epoll/kqueue/iocp →
Планировщик Go → Возобновление горутины

Как это работает:
1. Горутина вызывает Read/Write — FD регистрируется в netpoller
2. Горутина паркуется (gopark)
3. ОС сообщает, что fd готов — netpoller пробуждает горутину
4. Планировщик возвращает её в очередь выполнения

Преимущества:

— Тысячи соединений на одном потоке

— Без busy-wait, без блокировок

— Синхронный код — асинхронное исполнение

GOMAXPROCS

01 GOMAXPROCS определяет количество P в рантайме

02 runtime.GOMAXPROCS (число ядер процессора)

03 Увеличение GOMAXPROCS позволяет увеличить скорость выполнения CPU-bound операций, но не влияет на I/O bound, так как поток ОС в случае I/O будет занят

Livedemo

Каналы

Каналы связывают и синхронизируют горутины

```
ch := make(chan int)

go func() {
  ch <- 42 // отправка
}()

value := <-ch // получение
fmt.Println(value)
```

Канал — потокобезопасный способ обмена данными между горутинами.

У небуферизованного канала отправитель и получатель встречаются в точке передачи.

Оператор select

```
select {
case msg := <-ch1:
  fmt.Println("из ch1:", msg)
case msg := <-ch2:
  fmt.Println("из ch2:", msg)
default:
  fmt.Println("нет данных")
}
```

select ждёт готовности одной из операций.

Если готовы несколько — выбирает одну псевдослучайно.

default выполняется сразу, если остальные операции не готовы.

select как демультиплексор (loop with cancellation)

Создаём источник событий

```
func main() {
  data := make(chan int)
  quit := make(chan struct{})
  go func() {
    for i := 0; i < 5; i++ {
      data <- i
    }
    close(quit)
  }()
```

Продолжение main: слушаем каналы

```
  for {
    select {
    case v := <-data:
      fmt.Println("получено:", v)
    case <-quit:
      fmt.Println("завершение")
      return
    }
  }
}
```

select и graceful shutdown

```
func worker(ctx context.Context,
  jobs <-chan int) {
  for {
    select {
    case <-ctx.Done():
      return
    case job, ok := <-jobs:
      if !ok {
        return
      }
      fmt.Println("обработка:", job)
    }
  }
}
```

Горутина выходит при отмене контекста.

Закрытие jobs тоже завершает цикл: проверяем ok, чтобы не читать нулевые значения бесконечно.

Fan-out

Один канал задач — несколько воркеров

```
func worker(id int, jobs <-chan int,
  results chan<- int) {
  for job := range jobs {
    fmt.Println("worker", id, job)
    time.Sleep(time.Second)
    results <- job * 2
  }
}
```

Распределяем работу; ждём 5 результатов

```
func main() {
  jobs := make(chan int, 5)
  results := make(chan int, 5)
  for w := 1; w <= 3; w++ {
    go worker(w, jobs, results)
  }
  for j := 1; j <= 5; j++ {
    jobs <- j
  }
  close(jobs)
  for a := 1; a <= 5; a++ {
    fmt.Println(<-results)
  }
}
```

Fan-in

```go
func merge(cs ...<-chan int) <-chan int {
  var wg sync.WaitGroup
  out := make(chan int)
  output := func(c <-chan int) {
    defer wg.Done()
    for v := range c {
      out <- v
    }
  }
  wg.Add(len(cs))
  for _, c := range cs {
    go output(c)
  }
  go func() {
    wg.Wait()
    close(out)
  }()
  return out
}
```

```go
c1 := make(chan int)
c2 := make(chan int)
go func() {
  for i := 1; i <= 3; i++ {
    c1 <- i
  }
  close(c1)
}()
go func() {
  for i := 4; i <= 6; i++ {
    c2 <- i
  }
  close(c2)
}()
for n := range merge(c1, c2) {
  fmt.Println(n)
}
```

Pipeline

Каждый этап читает и создаёт канал

```
func double(in <-chan int) <-chan int {
  out := make(chan int)
  go func() {
    defer close(out)
    for n := range in {
      out <- n * 2
    }
  }()
  return out
}
```

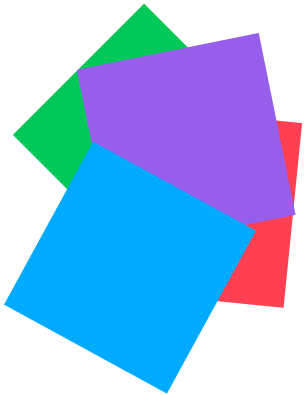
Pipeline: данные проходят этапы по очереди

```
// Источник → этап → потребитель
source := make(chan int)
go func() {
  defer close(source)
  for n := 1; n <= 3; n++ {
    source <- n
  }
}()
for n := range double(source) {
  fmt.Println(n) // 2, 4, 6
}
```

Типичные ошибки

Ошибка	Симптом	Решение
Чтение из закрытого канала	ok == false	Проверять второе значение
Отправка в закрытый канал	panic	Закрывает только отправитель
Блокировка навсегда	deadlock	Проверить порядок чтения / записи
Потерянные горутины	“утечки”	Использовать done или context
Чтение без default в select	зависание	Добавить таймаут
Голодание	простой горутин	Ограничить время выполнения или использовать очереди с приоритетом

Потренируемся



Какая проблема в этом коде?

```go
package main

import "fmt"

func f() {
  ch := make(chan int)
  go func() {
    data := <-ch
    fmt.Printf("Получил: %d\n", data)
  }()
  data := <-ch
  fmt.Printf("Main получил: %d\n", data)
}
```

Гонка данных?

Дедлок?

Голодание?

Что изменится, если вызвать эту функцию из main?

Какая проблема в этом коде?

1. Гонка данных
2. Дедлок
3. Голодание

```
func main() {
    var counter int
    var wg sync.WaitGroup

    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            counter++
            wg.Done()
        }()
    }

    wg.Wait()
    fmt.Printf("Counter: %d (expected: 1000)\n", counter)
}
```

Какая проблема в этом коде?

1. Гонка данных
2. Дедлок
3. Голодание

```
func main() {
    runtime.GOMAXPROCS(1)

    go func() {
        for i := 0; i < 5; i++ {
            fmt.Printf("горутина работает... %d\n", i)
            // Интенсивная работа без yield
            _ = make([]byte, 1024*1024)        }
    }()

    go func() {
        fmt.Println("горутина: жду своей очереди...")
        time.Sleep(100 * time.Millisecond)
        fmt.Println("горутина: выполнилась")
    }()

    time.Sleep(2 * time.Second)
}
```

Управление и завершение горутин

Инструменты управления горутинами:

- sync.WaitGroup
- contex.Context

sync.WaitGroup

```go
var wg sync.WaitGroup
for i := 0; i < 3; i++ {
  wg.Add(1)
  go func(i int) {
    defer wg.Done()
    fmt.Println("Worker", i)
  }(i)
}
wg.Wait()
```

Add(1) — увеличиваем счётчик до запуска горутины.

Done() — уменьшаем счётчик.

Wait() — ждём, пока он станет равен нулю.

Отмена контекста

Создаём контекст и передаём воркеру

```
ctx, cancel := context.WithCancel(
  context.Background())
defer cancel()
jobs := make(chan int)
go worker(ctx, jobs)
time.Sleep(time.Second)
cancel() // посылаем отмену
```

Воркер слушает канал отмены

```
func worker(ctx context.Context,
  jobs <-chan int) {
  for {
    select {
    case <-ctx.Done():
      return
    case job, ok := <-jobs:
      if !ok {
        return
      }
      fmt.Println(job)
    }
  }
}
```

Типичные ошибки

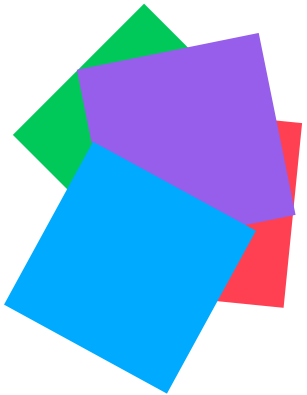
Ошибка	Почему плохо	Решение
Программа завершается до горутин	main завершилась первой	Использовать WaitGroup
Горутинная утечка	Горутина ждёт данные, которые никто не пришлёт	Закрывать каналы / передавать done
Состояние гонки	Несколько горутин пишут в общие переменные	Примитивы синхронизации (рассмотрим позже)
Забытый wg.Done()	Блокировка навсегда	Использовать defer wg.Done()

Вывод

Горутины — это мощный и легковесный инструмент обеспечения конкурентности в Go

В Go есть все инструменты для решения проблем конкурентности и управления горутинами, но использовать их нужно осознанно и с осторожностью

Ваши вопросы (5 минут)



III. Примитивы синхронизации, тикеры и таймеры

01 Примитивы синхронизации

02 Работа со временем в Go

03 Практическое применение

Примитивы синхронизации в Go

Примитив	Назначение
sync.Mutex	Взаимоисключение (один поток в критической секции)
sync.RWMutex	Несколько читателей, один писатель
sync.WaitGroup	Ожидание завершения горутин
sync.Once	Выполнить код ровно один раз
sync.Cond	Координация по событию

Mutex/RWMutex

Mutex: одна горутина в критической секции

```
var mu sync.Mutex
var count int

func increment() {
  mu.Lock()
  defer mu.Unlock()
  count++
}
```

RWMutex: читатели вместе, запись отдельно

```
var rw sync.RWMutex
var data = make(map[string]string)
func read(key string) string {
  rw.RLock()
  defer rw.RUnlock()
  return data[key]
}
func write(key, val string) {
  rw.Lock()
  defer rw.Unlock()
  data[key] = val
}
```

WaitGroup и Once

WaitGroup: ждём завершения группы

```go
var wg sync.WaitGroup
for i := 0; i < 3; i++ {
  wg.Add(1)
  go func(id int) {
    defer wg.Done()
    fmt.Println("worker", id)
  }(i)
}
wg.Wait()
```

Once: выполняем инициализацию один раз

```go
var once sync.Once

func initConfig() {
  once.Do(func() {
    fmt.Println("инициализация")
  })
}
```

sync.Cond

Ждём выполнения условия

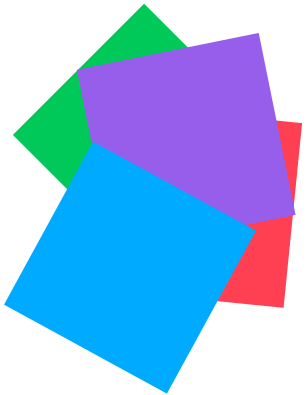
```
var mu sync.Mutex
cond := sync.NewCond(&mu)
ready := false
go func() {
  mu.Lock()
  for !ready {
    cond.Wait()
  }
  fmt.Println("Готово!")
  mu.Unlock()
}()
```

Меняем условие и посылаем сигнал

```
fmt.Println("Готовим...")
// Изменяем условие под мьютексом.
mu.Lock()
ready = true
mu.Unlock()
cond.Signal() // будим одного

// Wait освобождает мьютекс,
// а перед возвратом захватывает его.
// Условие проверяем в цикле:
// к этому моменту оно могло измениться.
```

Потренируемся



Вопрос

Как защитить общий запас томатов?

```
func makePizza() {
  tomatoCount := 10
  var pizzasMade int
  var wg sync.WaitGroup
  for i := 0; i < 5; i++ {
    wg.Add(1)
    go func(cookID int) {
      defer wg.Done()
      if tomatoCount > 0 {
        time.Sleep(10 * time.Millisecond)
        tomatoCount--
        pizzasMade++
```

Продолжение: ожидаем 5 пицц и 5 томатов

```
        fmt.Println(cookID, tomatoCount)
      } else {
        fmt.Println("Томаты закончились")
      }
    }(i)
  }
  wg.Wait()
  fmt.Println("Пицц:", pizzasMade)
  fmt.Println("Томатов:", tomatoCount)
}
```

Вопрос

```
func open() {
  fmt.Println("Пиццерия открывается")
  go func() {
    time.Sleep(100 * time.Millisecond)
    fmt.Println("Печь разогрета")
  }()
  go func() {
    time.Sleep(150 * time.Millisecond)
    fmt.Println("Ингредиенты готовы")
  }()
  go func() {
    time.Sleep(200 * time.Millisecond)
    fmt.Println("Персонал готов")
  }()
  fmt.Println("Принимаем заказы!")
}
```

Какой примитив поможет дождаться всех трёх подготовительных действий?

Где его нужно создать и какие операции добавить?

time.Timer

```
timer := time.NewTimer(3 * time.Second)

fmt.Println("Ожидание 3 секунд...")
<-timer.C
fmt.Println("Прошло 3 секунды!")
```

Timer срабатывает один раз.

Канал timer.C передаёт time.Time.

Чтение из канала блокирует горутину до срабатывания таймера.

Отмена таймера и сброс

Stop: отмена без зависшего чтения

```
timer := time.NewTimer(5 * time.Second)
defer timer.Stop()
event := make(chan struct{})
go func() {
  time.Sleep(2 * time.Second)
  close(event)
}()
select {
case <-event:
  timer.Stop()
  fmt.Println("Событие раньше таймера")
case <-timer.C:
  fmt.Println("Таймер сработал")
}
```

Reset: используем таймер повторно

```
timer := time.NewTimer(2 * time.Second)
<-timer.C
fmt.Println("Первое срабатывание")

timer.Reset(time.Second)
<-timer.C
fmt.Println("Второе срабатывание")

// Stop не закрывает канал C.
// После успешного Stop ждать из C
// без другого сигнала нельзя.
```

time.After

```
select {
case <-time.After(3 * time.Second):
  fmt.Println("Таймаут")
case res := <-response:
  fmt.Println("Ответ:", res)
}
```

After возвращает канал одноразового таймера.

Удобен для короткого ожидания в select.

Если нужны Stop или Reset — создаём Timer явно.

time.Ticker

```
ticker := time.NewTicker(time.Second)
defer ticker.Stop()

for i := 0; i < 5; i++ {
  t := <-ticker.C
  fmt.Println("Тик:", t.Format("15:04:05"))
}
```

Ticker периодически передаёт время в канал C.

Медленный получатель может пропускать тики.

Stop прекращает тики, но не закрывает канал. Выход из цикла организуем отдельно.

time.Ticker и select

Периодическая проверка + отмена

```
func monitor(ctx context.Context,
  name string) {
  ticker := time.NewTicker(2 * time.Second)
  defer ticker.Stop()
  for {
    select {
    case <-ctx.Done():
      fmt.Println(name, "остановлен")
      return
    case t := <-ticker.C:
      fmt.Println(name,
        t.Format("15:04:05"))
    }
  }
}
```

Запускаем мониторинг на 5 секунд

```
func main() {
  ctx, cancel := context.WithTimeout(
    context.Background(), 5*time.Second)
  defer cancel()
  go monitor(ctx, "service A")
  <-ctx.Done()
}

// Контекст ограничивает время работы.
// Горутина сама реагирует на отмену.
// Для ожидания её выхода можно
// дополнительно использовать WaitGroup.
```

Типичные ошибки

Ошибка	Что происходит	Решение
Нет выхода из цикла Ticker	Горутина продолжает работу	Отмена context + return
Используете time.After() в цикле	Создаёт кучу таймеров	Использовать time.NewTimer и Reset()
Чтение C после успешного Stop()	Ожидание без события	select с каналом отмены
Получатель Ticker не успевает	Часть тиков пропускается	Снизить частоту работы

Rate limiting

```
func processWithRateLimit(
  ctx context.Context, in <-chan int,
  burst int, refill time.Duration) {
  tokens := make(chan struct{}, burst)
  fill := func() {
    for len(tokens) < cap(tokens) {
      tokens <- struct{}{}
    }
  }
  fill()
  ticker := time.NewTicker(refill)
  defer ticker.Stop()
  for {
    select {
    case <-ctx.Done():
      return
    case <-ticker.C:
      fill()
    case job, ok := <-in:
      if !ok { return }
    waitToken:
      for {
        select {
        case <-ctx.Done():
          return
        case <-ticker.C:
          fill()
        case <-tokens:
          handle(job)
          break waitToken
        }
      }
    }
  }
}
```

Worker pool

```
func main() {
  jobs := make(chan job)
  results := make(chan result)
  const workers = 3
  var wg sync.WaitGroup
  wg.Add(workers)
  for w := 1; w <= workers; w++ {
    go worker(w, jobs, results, &wg)
  }
  go func() {
    for i := 1; i <= 10; i++ {
      jobs <- job{id: i,
        input: rand.Intn(10) + 1}
    }
    close(jobs)
  }()
```

```
  go func() {
    wg.Wait()
    close(results)
  }()
  for res := range results {
    if res.err != nil {
      fmt.Println(res.jobID, res.err)
      continue
    }
    fmt.Println(res.jobID, res.value)
  }
}
// Фрагмент запуска пула.
// worker читает jobs, пишет results
// и вызывает defer wg.Done().
// Канал закрывает владелец отправки.
```

Retry with deadline

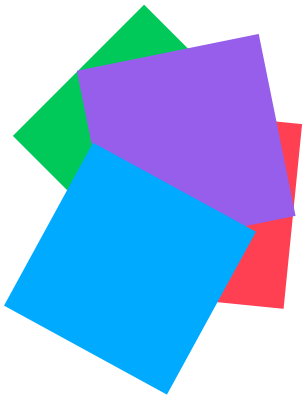
```go
func retryWithBackoff(ctx context.Context,
  maxAttempts int,
  initialDelay time.Duration,
  fn func(context.Context) error) error {
  delay := initialDelay
  for attempt := 1;
    attempt <= maxAttempts; attempt++ {
    if err := ctx.Err(); err != nil {
      return err
    }
    err := fn(ctx)
    if err == nil { return nil }
    if attempt == maxAttempts {
      return fmt.Errorf("предел: %w", err)
    }
    select {
    case <-ctx.Done():
      return ctx.Err()
    case <-time.After(delay):
    }
    delay = time.Duration(
      float64(delay) * 1.5)
    if delay > 800*time.Millisecond {
      delay = 800 * time.Millisecond
    }
  }
  return errors.New("нужно >= 1 попытки")
}
// Передаём ctx с deadline.
// fn тоже должна учитывать ctx.
```

Выводы

Примитивы синхронизации позволяют обеспечить безопасный конкурентный доступ к ресурсам, а таймеры и тикеры позволяют обеспечить периодичность действий

Инструмент	Поведение	Когда использовать
time.NewTimer(d)	Один сигнал через d	Таймаут или задержка
time.NewTicker(d)	Сигнал каждые d	Периодические действия
time.After(d)	Упрощённый Timer без Stop()	Одноразовые короткие таймауты
select + Timer	Управление временем ожидания	Таймаут при I/O
Ticker + context	Планировщик задач	Регулярные проверки и heartbeat

Ваши вопросы?



IV. Профилирование

01 Оптимизация и поиск проблем в конкурентном коде

Инструменты профилирования в Go

pprof

Встроенный инструмент профайлинга в Go

- blockprofile — места, где блокируются горутины
- mutexprofile — блокировки sync.Mutex
- goroutine — стеки горутин

```
import _ "net/http/pprof"



go tool pprof
http://localhost:3061/debug/pprof/profile
```

Инструменты профилирования в Go

```
import (
  "os"
  "runtime/trace"
)
func main() {
  f, err := os.Create("trace.out")
  if err != nil { panic(err) }
  defer f.Close()
  if err := trace.Start(f); err != nil {
    panic(err)
  }
  defer trace.Stop()
  // Здесь выполняется исследуемый код.
}
```

runtime/trace

Создание и блокировка горутин, планировщик, GC и системные вызовы.

Открыть трассировку:
go tool trace trace.out

Инструменты профилирования в Go

```
import (
  "testing"
  "go.uber.org/goleak"
)
func TestMain(m *testing.M) {
  goleak.VerifyTestMain(m)
}
// Или проверяем отдельный тест:
func TestSomething(t *testing.T) {
  defer goleak.VerifyNone(t)
  // Тестируем код с горутинами.
}
```

uber-go/goleak

Проверяет, что после тестов не осталось неожиданных горутин.

Показываем два способа подключения; выбираем подходящий для набора тестов.

Сегодня вы узнали, как:

01 Отличаются виды конкурентности

02 Использовать горутины, каналы, таймеры и тикеры

03 Корректно завершать работу горутин и отслеживать их состояние

Ваши вопросы?

